

Non-Convex Polygon Packing via Simulated Annealing and Bracketing-Bisection Search

Joel Solis*

April 2026

Abstract. The problem, featured in a 2024 Kaggle competition, is to find the minimum side length L^* of a square container that can hold N congruent copies without overlap, each free to translate and rotate. Answering this exactly is NP-hard. We describe a practical solver combining time-budgeted embarrassingly parallel Simulated Annealing with an outer bisection loop that narrows the bracket containing L^* , followed by a feasibility-polish stage. Collision detection uses the Separating Axis Theorem (SAT), chosen because it returns penetration depth, converting a combinatorial feasibility question into a smooth, navigable energy function. We deployed embarrassingly parallel jobs on a node of the HPC cluster for $N \in \{5, 10, 25, 50, 100, 200\}$, with wall times from 3 minutes to 8 hours. Packing densities range from $\eta \approx 0.58$ at small N to $\eta \approx 0.48$ at $N = 200$, with a $1/\sqrt{N}$ boundary-effect law supported by a geometric argument.

Keywords. polygon packing; simulated annealing; separating axis theorem; bisection search; embarrassingly parallel HPC; packing density scaling.

1. INTRODUCTION

Fix a non-convex polygon, a 15-vertex shape and make N identical copies. The problem is to find the smallest square that holds all of them without overlap. As N grows, the configuration space expands exponentially and exact enumeration becomes hopeless. Even for the simpler case of unit disks the placement decision problem is NP-complete [?]: if the decision version of a problem is NP-complete then its optimization version is NP-hard, and ours is at least as hard. Non-convexity adds a further layer of difficulty. It makes collision detection more expensive, the energy landscape more rugged and potentially fragmented into disconnected islands of feasible configurations.

Exact characterizations via the No-Fit Polygon (NFP), surveyed by Bennell and Oliveira [2], require convex decomposition for non-convex shapes and became for a fast prototyping study. We opted instead for a penalty-based Simulated Annealing (SA) approach, Penalty-based SA consistently outperforms approaches restricted to strictly feasible states: as Leung et al. [6] showed on orthogonal packing, hard feasibility constraints fragment the search space into disconnected islands, while continuous penalties create traversable bridges between

them. Genetic algorithms offer a compelling alternative by exploring structurally diverse configurations simultaneously [3], maintaining a population of qualitatively different solutions rather than a single chain. Parallel Tempering [?] is particularly interesting: it runs replicas at different temperatures and exchanges states between them via a Metropolis criterion, mixing exponentially faster than single-chain SA on multimodal landscapes. Both are valid and promising approaches that we leave as natural extensions of this work, given our time constraints.

For this quick prototyping study for a competition, SA by itself is the right starting point. As Sokal famously put it: “Monte Carlo is an extremely bad method; it should be used only when all alternative methods are worse.” What that observation misses is that its very simplicity makes it a reliable fallback that works across a broad class of problems. For us, SA is not a last resort so much as a deliberate foundation: it maps cleanly onto GPU architecture, its energy landscape connects naturally to statistical mechanics and the Ising model, and understanding its behavior on this geometry is the necessary groundwork before introducing the added complexity of population-based or replica methods.

2. PROBLEM FORMULATION

Let $\mathcal{P}$ be a fixed non-convex polygon with 15 vertices and area $A_p = 0.245625$, computed by the shoelace formula from the reference vertex array. Each copy i is placed by specifying its centre (x_i, y_i) and rotation angle $\theta_i \in [0, 2\pi)$; world vertices follow as $w_k^{(i)} = R(\theta_i) v_k + c_i$. A configuration is *feasible* if no two copies overlap and every copy lies within $\Omega_L = [-L/2, L/2]^2$. The goal is to find, for each N , the smallest L admitting a feasible configuration.

Since all N copies must fit inside Ω_L without overlap, the square area satisfies $L^2 \geq NA_p$, giving the hard lower bound $L_{\text{area}} = \sqrt{NA_p}$; the bisection loop never probes below this. Our Simulated Annealing implementation performs a move by selecting a random copy i and proposing a new state (x'_i, y'_i, θ'_i) by adding a random offset to the current state, with step sizes (s_{xy}, s_θ) that adapt over time to maintain a target acceptance rate. The energy change ΔE is computed incrementally by checking only pairs involving copy i , and the move is accepted with probability $\min(1, e^{-\Delta E/T})$ where T is the current temperature. If the energy is within the feasibility threshold, the configuration is saved and the bisection loop updates its bracket on L^*

*Department of Mathematics, University of Arizona. jsolis@math.arizona.edu. Supported by allocation ece569 on the University of Arizona HPC cluster.

accordingly. After bisection, an adaptive polish phase continues to attempt smaller L values until the wall time budget is exhausted, with the best known L^* updated on every improvement. Restricting the search to strictly non-overlapping configurations at all times confines it to disconnected feasible islands in $\mathbb{R}^{2N} \times [0, 2\pi)^N$. Allowing temporary, penalized violations creates navigable paths between those islands, which is why the choice of a continuous collision metric rather than a binary one matters so much.

3. ALGORITHM

3.1. Collision Detection Pipeline

During our Simulated Annealing pipeline, collision detection is invoked after every proposed move to evaluate the energy change, making it the dominant runtime cost. Three collision detection mechanisms are employed: Boundin circle checks, axis-aligned bounding box (AABB) checks, and the Separating Axis Theorem (SAT). In this order each detection becomes more expensive. Having a mixture of cheap and expensive checks allows us to filter most detections early, so that the expensive SAT is only invoked on a small fraction of pairs that pass the broad-phase filters. Naive pairwise checking costs $O(N^2)$ per iteration. A uniform spatial hash grid with cell size double the size of the bounding radius $h = 2r_{br} = 1.6$ restricts each query to a 5×5 neighbourhood of $\approx 261\eta$ candidates, where $\eta = NA_p/L^2$ is the packing density. The energy change is then computed incrementally, touching only pairs involving the moved polygon, reducing the effective per-move cost to $O(1)$. Of those candidates, A large fraction of these candidates is expected to be discarded by the polygon-level AABB checks. Group-level and per-triangle AABB checks prune the remainder before the Separating Axis Theorem (SAT) is invoked. Figure 1 visualizes each pahse of the four-stage collision detections pipeline.

Offline, the polygon is triangulated as a fan from vertex 0 into $T = 13$ triangles. These are grouped into three clusters of sizes $(5, 5, 3)$ and cached as AABBs. During narrow-phase evaluation, the SAT projects triangle pairs onto six edge-normal axes to compute the minimum overlap width, $d \geq 0$. World-space vertices are projected directly to avoid recomputing rotation matrices on each call. If a gap appears on any axis, the triangles are disjoint. Otherwise, $d > 0$ yields the penetration depth, providing a continuous measure of intersection severity. Relying on a binary collision penalty would create a flat energy landscape with no gradient. By contrast, the depth d supplies a clear directional signal. Then, feeding d^2 into the energy function generates the smooth guidance necessary to steer the SA optimization toward feasibility from arbitrary starting states.

3.2. Energy Function and SA Schedule

We model feasibility as a soft-constrained optimization problem, penalizing both geometric interpenetration and container boundary violations. The total energy of a configuration at container side length L is

$$E = \lambda \sum_{i < j} \sum_{(a,b) \in \mathcal{T}_{ij}} d(a,b)^2 + \mu \sum_i \phi_i(L),$$

Table 1. Two-phase SA schedule parameters.

Parameter	Phase A	Phase B
Iterations M	70,000	35,000
T_{start}	0.30	0.06
T_{end}	3×10^{-4}	10^{-6}
Overlap power p	1	2
λ_0, μ_0	10^2 (fixed)	10^3 (ramped)
Step adapt window	1,500 iters	
Acceptance band	[0.25, 0.55]	[0.10, 0.30]
Step shrink / grow	0.88 / 1.12	0.90 / 1.08
Reinsertion prob.	2%	0.5%
Feasibility $\mathcal{F} <$	10^{-6}	

where $d(a,b)$ is the SAT penetration depth between triangle pair (a,b) , $\phi_i(L)$ is the squared AABB boundary violation of copy i , and (λ, μ) are the respective penalty weights. A configuration is declared strictly feasible when the unweighted residual is negligible:

$$\mathcal{F} = \sum_{i < j} \text{overlap}_{ij} + \sum_i \phi_i < 10^{-6}.$$

To minimize E , the SA trial executes two sequential phases with geometrically decreasing temperature $T_k = T_0 \beta^k$, where $\beta = (T_{\text{end}}/T_0)^{1/M}$: Phase A (Exploration; 70,000 iterations): Prioritizes broad search using a high initial temperature $T_0 = 0.30$ and a linear overlap penalty ($p = 1$). Phase B (Feasibility; 35,000 iterations): Drives the system toward strict feasibility by dropping T_0 to 0.06, switching to a quadratic penalty ($p = 2$), and scaling penalty weights by a factor of 1.8 every 1,200 iterations. Throughout both phases, step sizes (s_{xy}, s_θ) dynamically adapt every 1,500 iterations to maintain a target acceptance band, automatically coupling the search radius to the cooling temperature. While small N instances execute the full 105,000-iteration budget, wall-clock limits may truncate Phase B for larger N . Table 1 summarizes the schedule.

3.3. Outer Loop: Bisection and Polish

The SA oracle answers one question: is there a feasible configuration at this L ? Finding the minimum L requires an outer bisection loop. Starting from an initial bracket $[L_{\text{lo}}, L_{\text{hi}}]$ with $L_{\text{lo}} = L_{\text{area}}$ and $L_{\text{hi}} = \gamma L_{\text{lo}}$ ($\gamma \in \{2.2, 2.6, 3.0\}$ depending on N), the algorithm evaluates $L_{\text{mid}} = \frac{1}{2}(L_{\text{lo}} + L_{\text{hi}})$ at each step: a feasible result sets $L_{\text{hi}} \leftarrow L_{\text{mid}}$, an infeasible one sets $L_{\text{lo}} \leftarrow L_{\text{mid}}$. The loop is configured for up to 26 steps; in practice, wall-clock time limits may terminate it earlier, particularly at large N . Before each SA run at a new L , all polygon centres are rescaled by $\alpha = L_{\text{new}}/L_{\text{old}}$, a warm start that preserves structural information and reduces the time to re-establish feasibility.

After bisection, an adaptive polish phase progressively shrinks the container: at each attempt the solver tries $L \leftarrow L^*(1 - \varepsilon)$ with ε adapted over rolling windows of 20 probes to maintain a target success rate of roughly 35%, then runs SA at the candidate L . A successful packing updates the best known L^* . Artifacts confirm this phase produces repeated improvements in L over time, and it accounts for the majority of density

gain at large N . Progress is written to disk on every improvement, and a global snapshot is flushed on SIGTERM so SLURM preemption loses no work. Every worker is seeded deterministically from (base seed, run ID, trial index) via SplitMix64, guaranteeing full reproducibility.

4. EXPERIMENTAL SETUP

All computational experiments were conducted on the University of Arizona high-performance computing (HPC) cluster. The results reported herein derive from *single-node SLURM array jobs*. Each array task executed a single independent random seed, with a maximum of two seeds running concurrently per job. We configured the array sizes and wall-clock limits according to the problem size: 8 tasks at $N = 25$ (4-hour wall time, yielding an effective search budget of 14,100,s after a 300-second post-processing cushion), 6 tasks at $N = 50$ (4,h), 4 tasks at $N = 100$ (4,h), and 2 tasks at $N = 200$ (8,h wall time, yielding a 28,500,s effective budget). To ensure data integrity, SLURM was configured to send a SIGTERM signal 120,s prior to the hard time limit, allowing the solver sufficient time to flush its best-found snapshot to disk. Because these runs executed without inter-worker communication, the global best solution for each instance of N was identified through a post-hoc reduction pass over all resulting output CSV files.

5. RESULTS

Table 2 reports the best feasible side length L^* and packing efficiency $\eta = NA_p/(L^*)^2$ for each N . The area lower bound $L_{\text{area}} = \sqrt{NA_p}$ is given for reference; $\eta = 1$ would require zero wasted space, which is geometrically impossible for this non-convex shape. The column “Runs” counts workers that found at least one feasible configuration.

Table 2. Best feasible side length, packing efficiency, and feasibility rate by instance size. $N = 5$ and $N = 10$ used the five-node pipeline (3-min smoke tests); $N \geq 25$ used single-node array jobs (4–8 h). Cross- N comparisons should be read with this provenance difference in mind.

N	L_{area}	L^*	η	Runs	t_{first}
5	1.108	1.472	0.567	15	5 s
10	1.568	2.056	0.581	15	6 s
25	2.478	3.315	0.559	6	8 s
50	3.504	4.652	0.567	4	46 s
100	4.956	6.785	0.533	3	53 s
200	7.009	10.104	0.481	1	19 s

Three patterns emerge. First, the number of workers reaching feasibility drops sharply: 15 of 15 at $N = 5$ and $N = 10$, down to 6 at $N = 25$, and to a single run at $N = 200$. The SA is time-budgeted and not iteration-bounded, but the cost per SA proposal grows with N because more polygon pairs must be checked. At large N , fewer trials fit within the wall-time budget, and the trials that do run explore a configuration space of dimension $3N$ with proportionally less coverage per polygon. The configuration space at $N = 200$ has 600 degrees of freedom; the budget was calibrated on much smaller instances.

Second, best-of-best efficiency η declines monotonically, from 0.58 at small N to 0.48 at $N = 200$. Fitting $\eta(N) \approx \eta_\infty + c/\sqrt{N}$ to the $N = 50$ and $N = 200$ points gives $\eta_\infty \approx 0.40$ and $c \approx 1.22$. The scaling has a clean physical basis: wasted space due to boundary proximity grows with the container perimeter, which scales as $L \propto \sqrt{N}$, while bulk packing capacity grows with the area, scaling as $L^2 \propto N$; the relative boundary penalty therefore decays as $1/L \propto 1/\sqrt{N}$. The extrapolated limit $\eta_\infty \approx 0.40$ should be read with caution: at large N the density drop reflects solver budget limitations as much as any geometric asymptote.

Third, the median time to final best configuration was 205 s at $N = 25$ but exceeded 3,250 s at $N = 100$, out of a 14,100 s budget. Most improvement arrived at 23–26% into the run, confirming that the polish phase, not the bisection loop, produces the majority of the density gain at large N . At small N the bisection alone is nearly sufficient; at large N the bulk of the productive computation happens during extended feasibility polish.

6. CONCLUSION

Choosing SAT over a binary overlap test was the key design decision: penetration depth provides the continuous energy gradient that steers SA toward feasibility from arbitrary, heavily overlapping initial configurations. The bisection loop narrows the bracket on L^* over up to 26 steps, and the adaptive polish phase then shrinks the container further by progressively attempting smaller L values, filling whatever wall time remains. Run artifacts confirm that this polish stage produces the majority of the post-bisection density gain, particularly at large N . The experiments confirm the solver reaches feasible packings for N up to 200 within practical HPC allocations, but expose a clear bottleneck: as N grows, the time-budgeted SA visits fewer trials relative to the dimension of the search space, and the feasibility rate collapses. The natural first intervention is to scale the time budget proportionally with N .

Three directions motivate future work. A CUDA implementation targeting the P100 (compute capability 6.0) could exploit the embarrassingly parallel trial structure, with one thread block per polygon pair and triangle vertex arrays in shared memory. Parallel Tempering, which runs replicas at a geometric temperature ladder and proposes Metropolis swaps between adjacent temperatures, could seed cold-chain refinement from configurations that single-chain SA never visits, though it requires substantially longer wall times than tested here. Finally, a genetic algorithm treating full polygon layouts as chromosomes could maintain structural diversity across qualitatively different configurations simultaneously, something serial SA cannot do.

REFERENCES

- [1] Kaggle. Santa 2025 competition: Christmas Tree Packing Challenge kaggle.com/competitions/santa-2025, 2025.
- [2] J. A. Bennell and J. F. Oliveira. The geometry of nesting problems: A tutorial. *Eur. J. Oper. Res.*, 184(2):397–415, 2008.

- [3] E. K. Burke, G. Kendall, and G. Whitwell. A new placement heuristic for the orthogonal stock-cutting problem. *Oper. Res.*, 52(4):655–671, 2004.
- [4] Kaggle. Santa 2025 competition: Christmas Tree Packing Challenge. kaggle.com/competitions/santa-2025, 2025.
- [5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [6] T. W. Leung, C. K. Chan, and M. D. Troutt. Application of a mixed simulated annealing-genetic algorithm heuristic for the two-dimensional orthogonal packing problem. *Eur. J. Oper. Res.*, 145(3):530–542, 2003.
- [7] A. Sokal. Monte Carlo methods in statistical mechanics: Foundations and new algorithms. In *Functional Integration*, pages 131–192. Springer, 1997.
- [8] T. Vogel and Y. W. Li. Accelerating multicanonical sampling with irreversibility. *arXiv preprint arXiv:2601.18980*, 2026.
- [9] L. Zhang, P. Potapchik, J. He, Y. Du, A. Doucet, F. Vargas, H.-D. Dau, and S. Syed. Accelerated parallel tempering via neural transports. *arXiv preprint arXiv:2502.10328*, 2025.

Collision Detection Pipeline

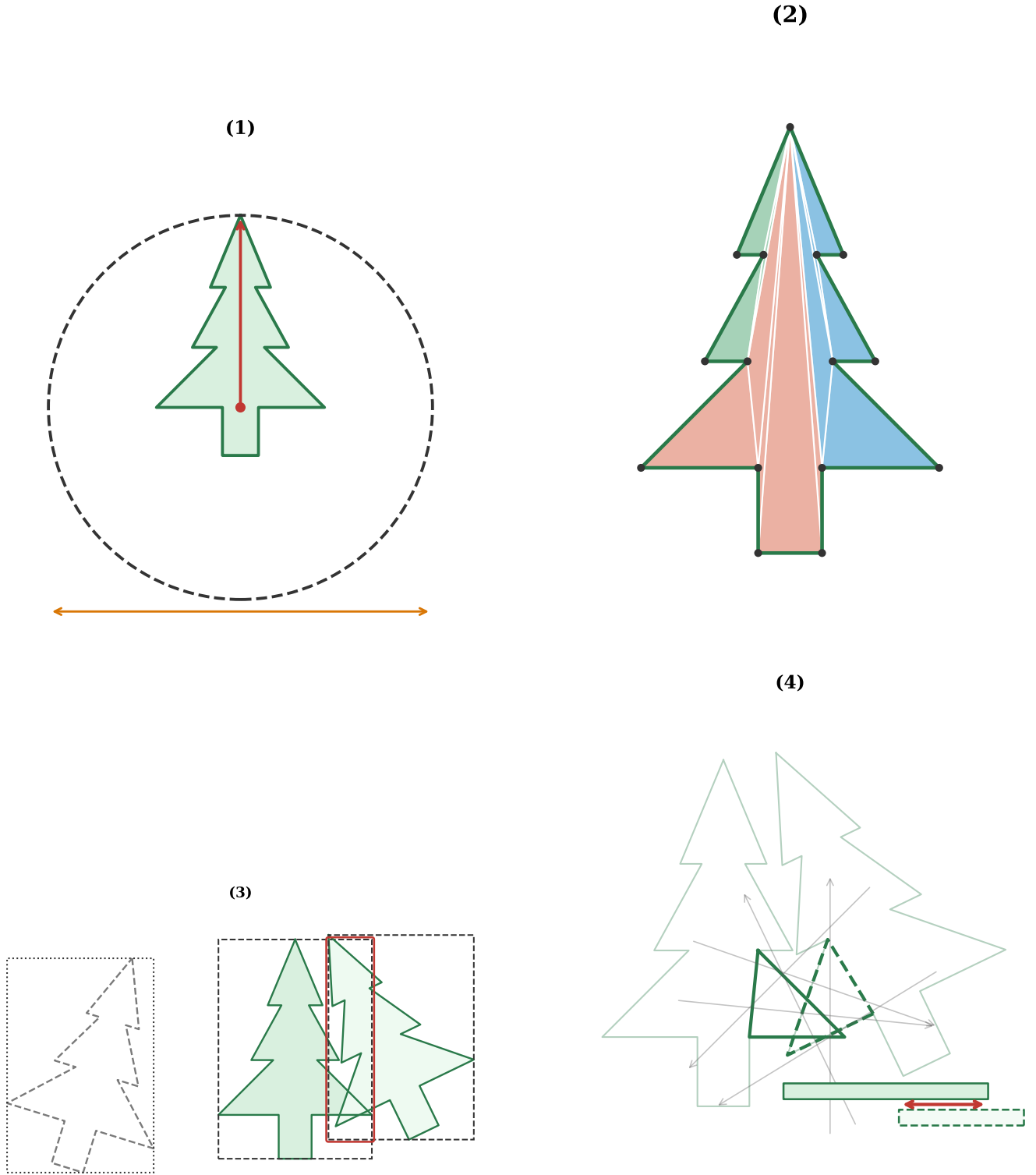


Figure 1. Collision detection pipeline. Top-left: broad phase using a uniform spatial hash and neighborhood query. Top-right: polygon AABB culling. Bottom-left: group-level AABB pruning after triangulation into clustered subsets. Bottom-right: triangle-triangle SAT narrow phase returning penetration depth d , which feeds the overlap energy.

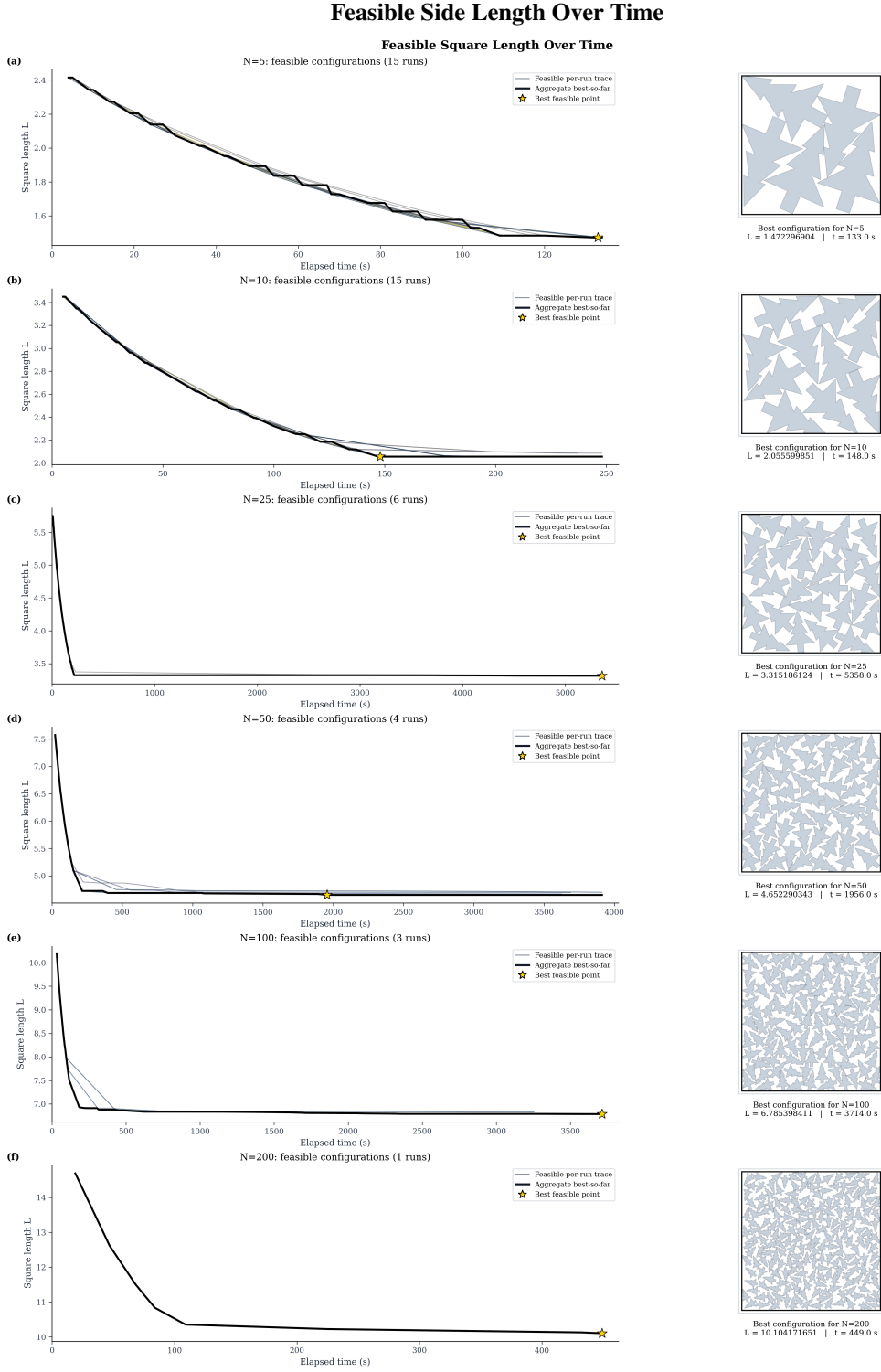


Figure 2. Feasible square side length L versus wall time across $N \in \{5, 10, 25, 50, 100, 200\}$. The traces show that large- N improvements arrive later and that the search spends much longer near its best-known feasible bracket before finding further gains.

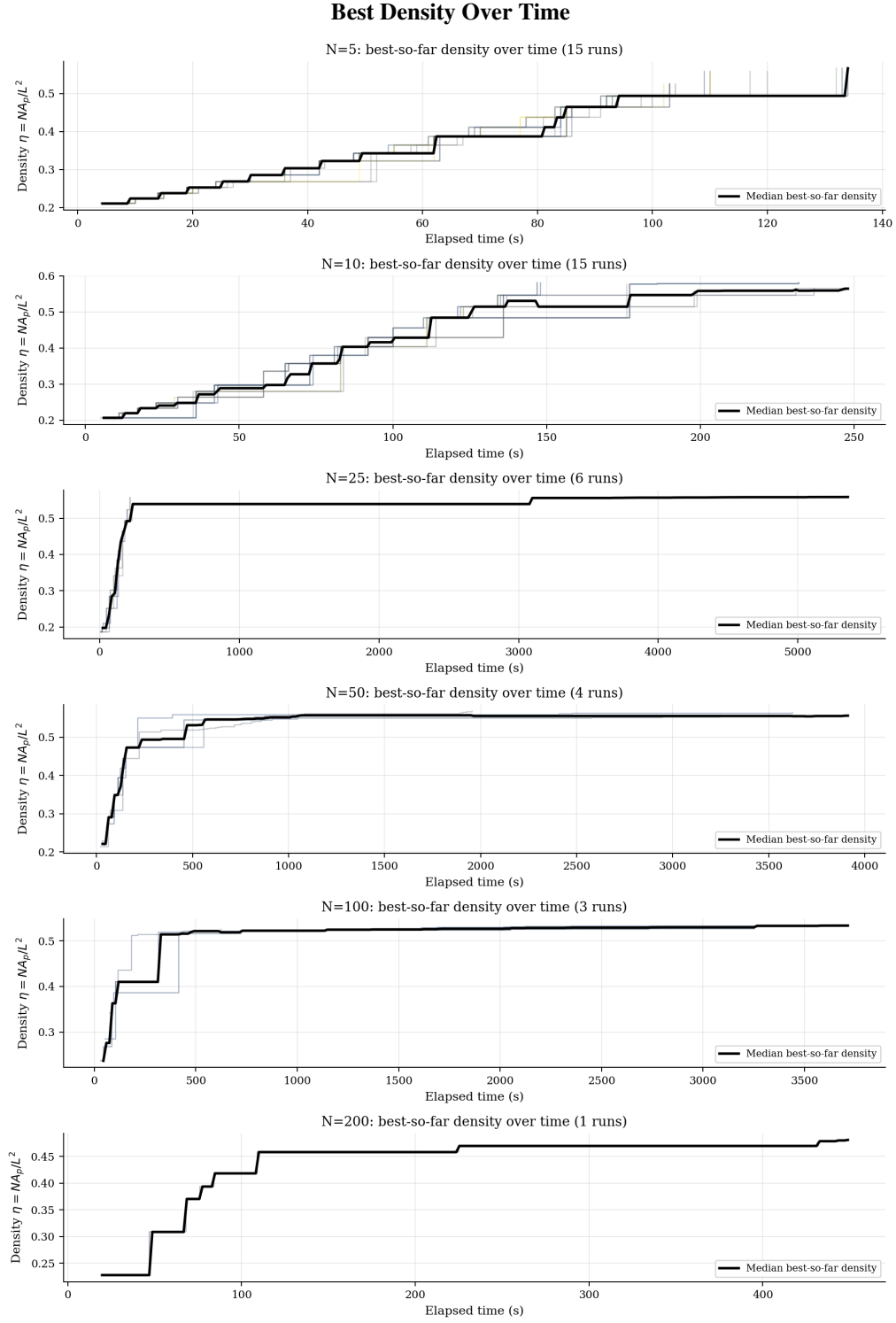


Figure 3. Best-so-far packing density $\eta = NA_p/L^2$ versus wall time. The late-time slope at larger N is consistent with most of the final improvement occurring during feasibility polish rather than during the initial bracket search.